CS223 Computer Architecture & Organization

Performance Evaluation Methods

John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Measuring Performance

- **When can we say one computer / architecture / design is better than others?**
  - Desktop PC – (execution time of a program)
  - Server (transactions / unit time)

- **When can we say X is n times faster than Y ?**
  - Execution time$_Y$ / Execution time$_X$ =n
  - Throughput $_x$/ Throughput $_y$ =n

Measuring Performance

- **Typical performance metrics:**
  - Response time
  - Throughput
  - CPU time
  - Speedup
- **Benchmarks**
  - Toy programs (e.g. sorting, matrix multiply)
  - Synthetic benchmarks (e.g. Dhrystone)
  - Benchmark suites (e.g. SPEC06, SPLASH)

Benchmark Suite

SPEC CPU 2017 has 43 benchmarks: https://www.spec.org/cpu2017/

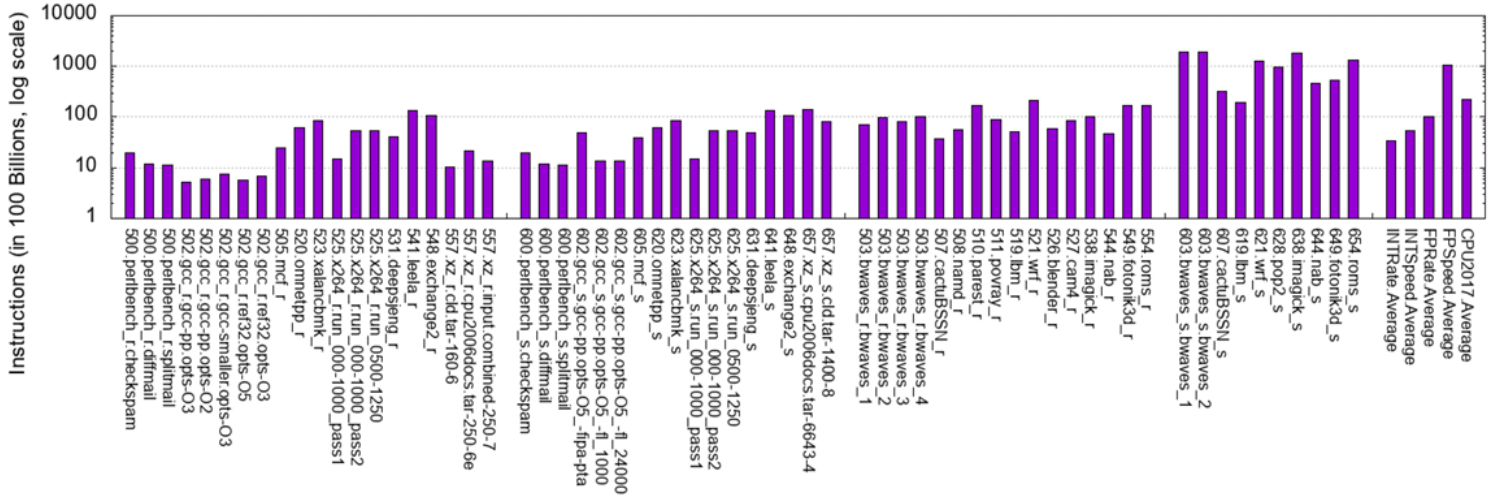
SPECspeed®2017 Integer	Language [1]	KLOC [2]	Application Area
600.perlbench_s	C	362	Perl interpreter
602.gcc_s	C	1,304	GNU C compiler
605.mcf_s	C	3	Route planning
620.omnetpp_s	C++	134	Discrete Event simulation - computer network
623.xalancbmk_s	C++	520	XML to HTML conversion via XSLT
625.x264_s	C	96	Video compression
631.deepsjeng_s	C++	10	Artificial Intelligence: alpha-beta tree search (Chess)
641.leela_s	C++	21	Artificial Intelligence: Monte Carlo tree search (Go)
648.exchange2_s	Fortran	1	Artificial Intelligence: recursive solution generator (Sudoku)
657.xz_s	C	33	General data compression

Benchmark Suite

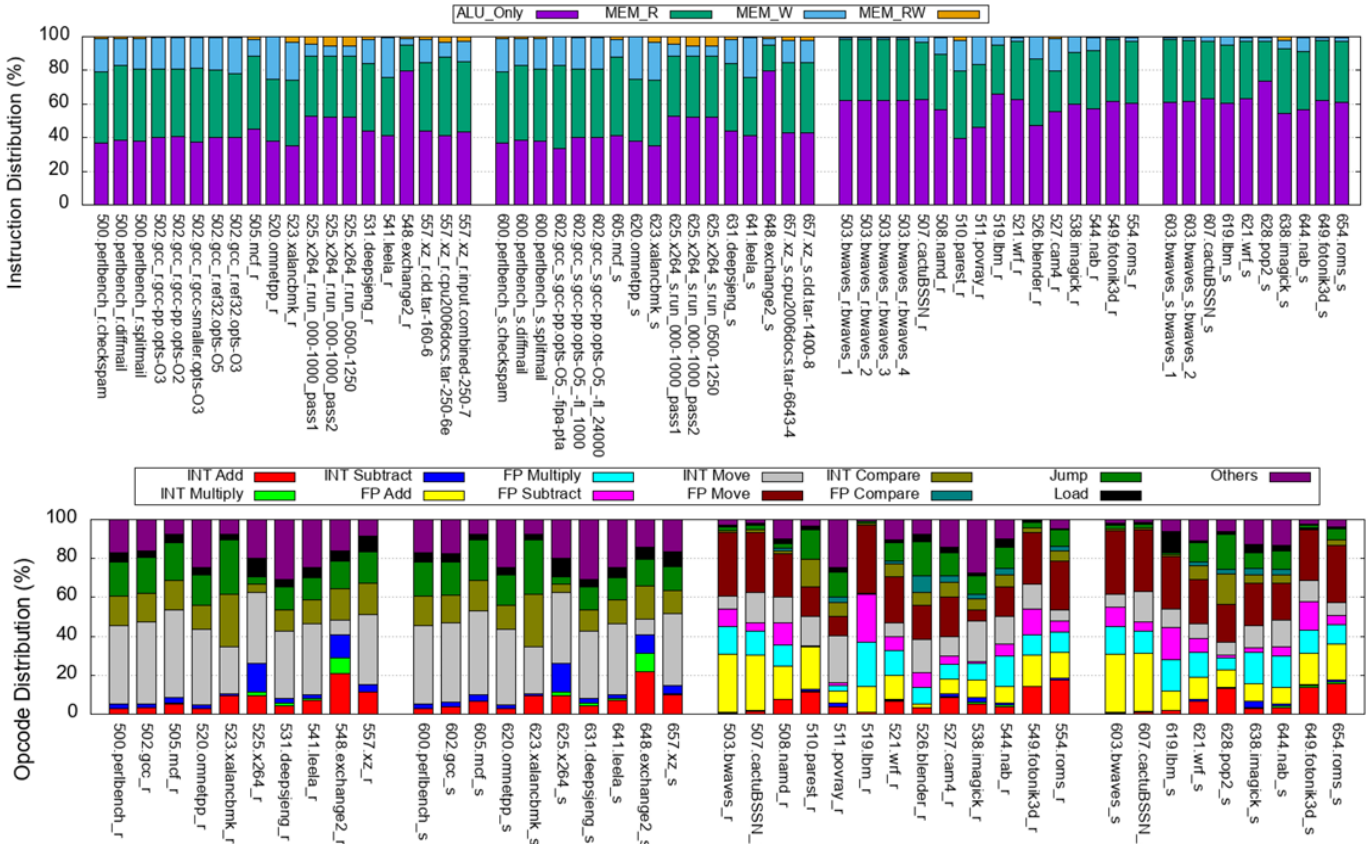
SPEC CPU 2017 has 43 benchmarks: https://www.spec.org/cpu2017/

SPECrate®2017 Floating Point	SPECspeed®2017 Floating Point	Language [1]	KLOC [2]	Application Area
503.bwaves_r	603.bwaves_s	Fortran	1	Explosion modeling
507.cactuBSSN_r	607.cactuBSSN_s	C++, C, Fortran	257	Physics: relativity
508.namd_r		C++	8	Molecular dynamics
510.parest_r		C++	427	Biomedical imaging: optical tomography with finite elements
511.povray_r		C++, C	170	Ray tracing
519.lbm_r	619.lbm_s	C	1	Fluid dynamics
521.wrf_r	621.wrf_s	Fortran, C	991	Weather forecasting
526.blender_r		C++, C	1,577	3D rendering and animation
527.cam4_r	627.cam4_s	Fortran, C	407	Atmosphere modeling
	628.pop2_s	Fortran, C	338	Wide-scale ocean modeling (climate level)
538.imagick_r	638.imagick_s	C	259	Image manipulation
544.nab_r	644.nab_s	C	24	Molecular dynamics
549.fotonik3d_r	649.fotonik3d_s	Fortran	14	Computational Electromagnetics
554.roms_r	654.roms_s	Fortran	210	Regional ocean modeling

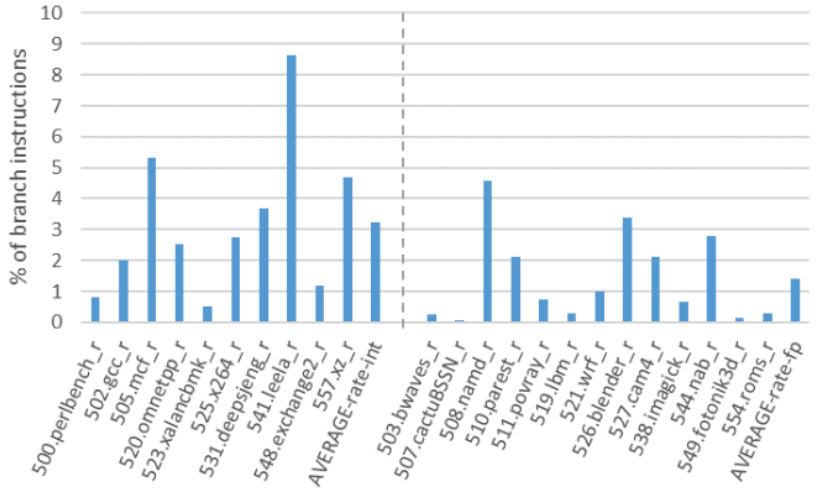
Benchmark Based Evaluation

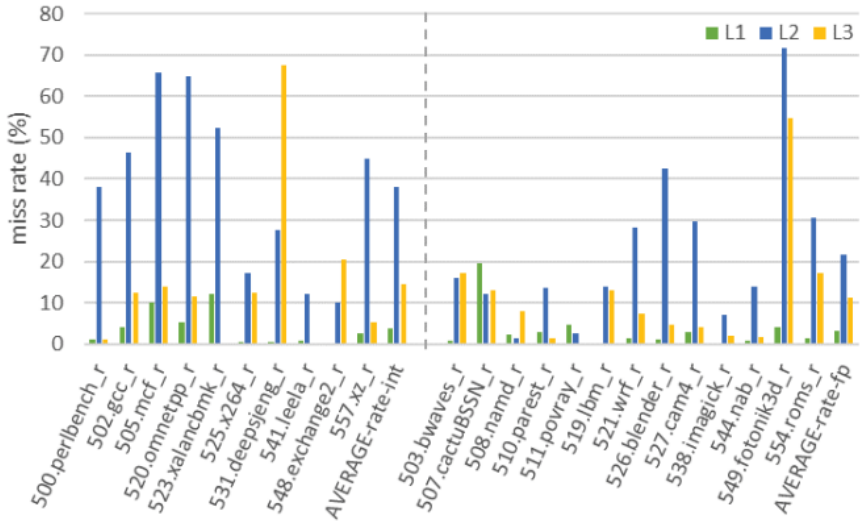


Benchmark Based Evaluation



Benchmark Based Evaluation





SPEC Ratio

$$\text{SPECRatio}_A = \frac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_A}$$

Reference for SPEC 2017:
Sun Fire V490 with 2100 MHz UltraSPARC-IV

$$\frac{\text{SPECRatio}_A}{\text{SPECRatio}_B} = \frac{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_A}}{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_B}} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{\text{Performance}_A}{\text{Performance}_B}$$

$$GeometricMean\,(a_1, a_2, a_3, \ldots, a_N) = \sqrt[N]{\prod_i^N a_i}$$

Amdahl's Law

- Amdahl's Law defines the speedup that can be gained by improving some portion of a computer.
- The performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used.

$$ET_{new} = ET_{old} \times (1 - fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}$$

$$Speedup = \frac{ET_{old}}{ET_{new}} = \frac{1}{(1 - fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}}$$

Amdahl's Law- Illustration

Example: Suppose that we want to enhance the floating point operations of a processor by introducing a new advanced FPU unit. Let the new FPU is 10 times faster on floating point computations than the original processor. Assuming a program has 40% floating point operations, what is the overall speedup gained by incorporating the enhancement?

Solution:

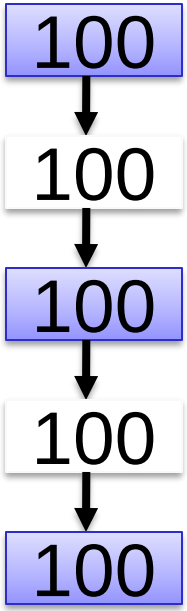
Fraction enhanced = 0.4

Speedup enhanced = 10

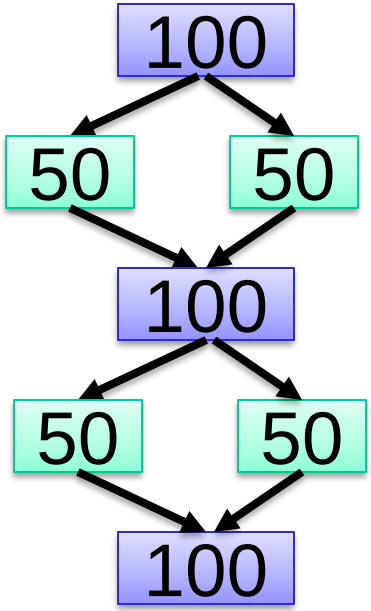
$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

$$\text{Speedup}_{\text{overall}} = \frac{1}{0.6 + \frac{0.4}{10}} = \frac{1}{0.64} \approx 1.56$$

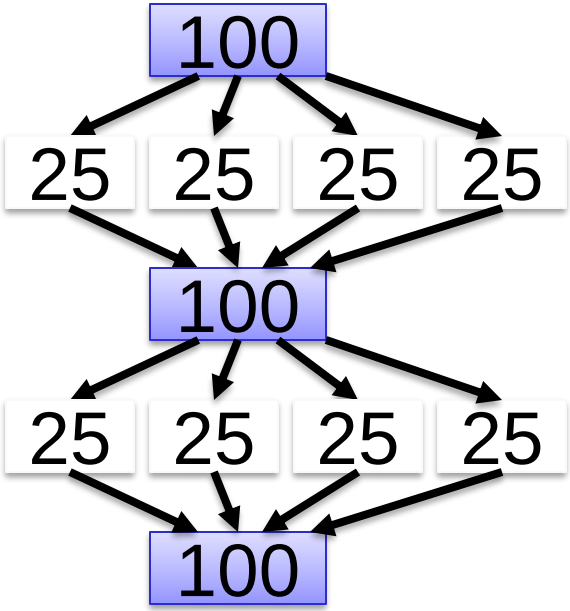
Amdahl's Law for Parallel Processing



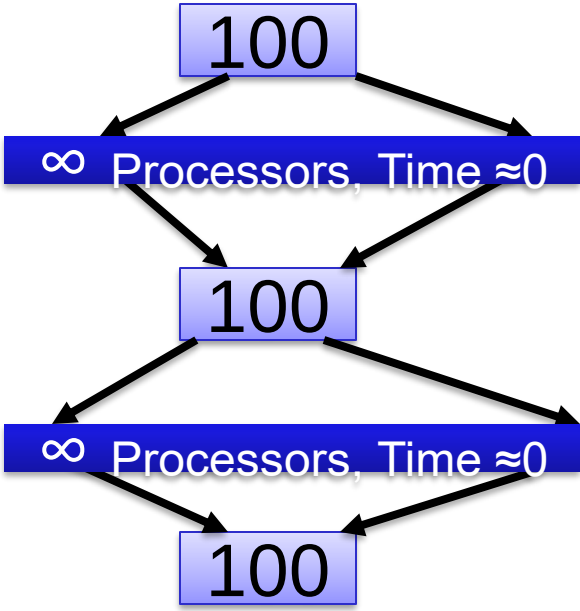
Work 500,
Time 500
Sp=1X



Work 500,
Time 400
Sp=1.25X



Work 500,
Time 350
Sp=1.4X

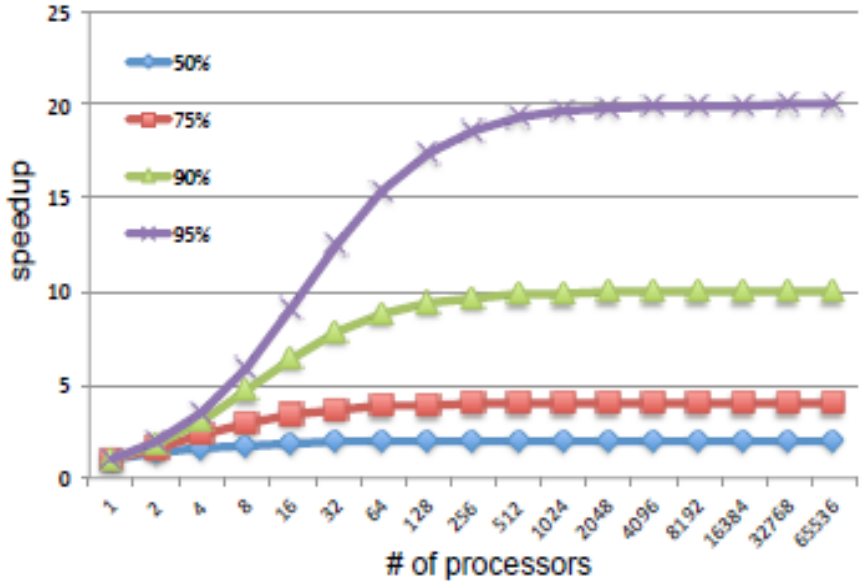


Work 500,
Time 300
Sp=1.7X

How much Speed up you can achieve ?

Amdahl's Law:

$$Speedup = \frac{1}{(1-\alpha) + \frac{\alpha}{n}}$$



Design Example

A common transformation required in graphics processors is square root. Implementations of floating-point (FP) square root vary significantly in performance, especially among processors designed for graphics. Suppose FP square root (FPSQR) is responsible for 20% of the execution time of a critical graphics benchmark.

One proposal is to enhance the FPSQR hardware and speed up this operation by a factor of 10. The other alternative is just to try to make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application. Compare these two design alternatives using Amdahl's Law.

Design Example

Case A: FPSQR hardware optimization

$$S = \frac{1}{(1-f) + \frac{f}{N}}$$

$$\frac{1}{(1-0.2) + \frac{0.2}{10}} = \frac{1}{(0.8) + 0.02}$$

$$= 1.219 \text{ times.}$$

Case B: FP instructions optimization

$$S = \frac{1}{(1-f) + \frac{f}{N}}$$

$$\frac{1}{(1-0.5) + \frac{0.5}{1.6}} = \frac{1}{(0.5) + 0.3125}$$

$$= 1.23 \text{ times.}$$

Principles of Computer Design

- All processors are driven by clock.
- Expressed as clock rate in GHz or clock period in ns
- CPU Time = CPU clock cycles x clock cycle time

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction Count}}$$

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instructions}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

Principles of Computer Design

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instructions}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

- Clock cycle time- hardware technology
- CPI- organization and ISA
- IC-ISA and compiler technology

Principles of Computer Design

- Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum_{i=1}^{n} \text{ICi} \times \text{CPI}_i$$

$$\text{CPU time} = \left(\sum_{i=1}^{n} \text{ICi} \times \text{CPI}_i \right) \times \text{CCT}$$

Example: Basic Performance Analysis

Consider two programs A and B that solves a given problem. A is scheduled to run on a processor P1 operating at 1 GHz and B is scheduled to run on processor P2 running at 1.4 GHz. A has total 10000 instructions, out of which 20% are branch instructions, 40% load store instructions and rest are ALU instructions. B is composed of 25% branch instructions. The number of load store instructions in B is twice the count of ALU instructions. Total instruction count of B is 12000. In both P1 and P2 branch instructions have an average CPI of 5 and ALU instructions has an average CPI of 1.5. Both the architectures differ in the CPI of load-store instruction. They are 2 and 3 for P1 and P2, respectively. Which mapping (A on P1 or B on P2) solves the problem faster, and by how much?

Example: Basic Performance Analysis

A on P1 (1GHz ☐ CCT = 1ns)	B on P2 (1.4 GHz☐ CCT = 0.714ns)
IC=10000	IC=12000
Fraction BR: L/S: ALU = 20: 40: 40	Fraction BR: L/S: ALU = 25: 50: 25
CPI of BR: L/S: ALU = 5: 2: 1.5	CPI of BR: L/S: ALU = 5: 3 : 1.5

(a) CPI A_P1=(0.2x5 + 0.4x2 + 0.4x1.5) = 2.4

ExT = 2.4 x10000x1 ns= **24000 ns**

(b) CPI B_P2=(0.25x5 + 0.5x3 + 0.25x1.5) = **3.125**

ExT = 3.125 x12000x0.714 ns = **26775 ns**

Hence A on P1 is faster.

Example: Amdahl's Law

A company is releasing 2 latest versions (beta and gamma) of its basic processor architecture named alpha. Beta and gamma are designed by making modifications on three major components (X, Y and Z) of the alpha. It was observed that for a program A the fractions of the total execution time on these three components, X, Y, and Z are 40%, 30%, and 20%, respectively. Beta speeds up X and Z by 2 times but slows down Y by 1.3 times, where as gamma speeds up X, Y and Z by 1.2, 1.3 and 1.4 times, respectively.

(a) How much faster is gamma over alpha for running A?

(b) Whether beta or gamma is faster for running A? Find the speedup factor

$$S = \frac{1}{(1 - fx - fy - fz) + \left(\frac{fx}{Nx} + \frac{fy}{Ny} + \frac{fz}{Nz} \right)}$$

fx=0.4 : fy=0.3: fz=0.2

Beta: Nx=2 : Ny=1/1.3 : Nz=2

Gamma: Nx=1.2 : Ny=1.3 : Nz=1.4

Example: Amdahl's Law

$$S = \frac{1}{(1 - fx - fy - fz) + \left(\frac{fx}{Nx} + \frac{fy}{Ny} + \frac{fz}{Nz}\right)}$$

$$S\_beta/alpha = \frac{1}{(0.1) + \left(\frac{0.4}{2} + \frac{0.3}{0.77} + \frac{0.2}{2}\right)} = 1.267 \text{ times}$$

$$S\_gamma/alpha = \frac{1}{(0.1) + \left(\frac{0.4}{1.2} + \frac{0.3}{1.3} + \frac{0.2}{1.4}\right)} = 1.239 \text{ times}$$

(a) Gamma is 1.239 times faster over alpha

(b) Beta is faster than gamma→.267/1.239 = 1.022 times

johnjose@iitg.ac.in
http://www.iitg.ac.in/johnjose/